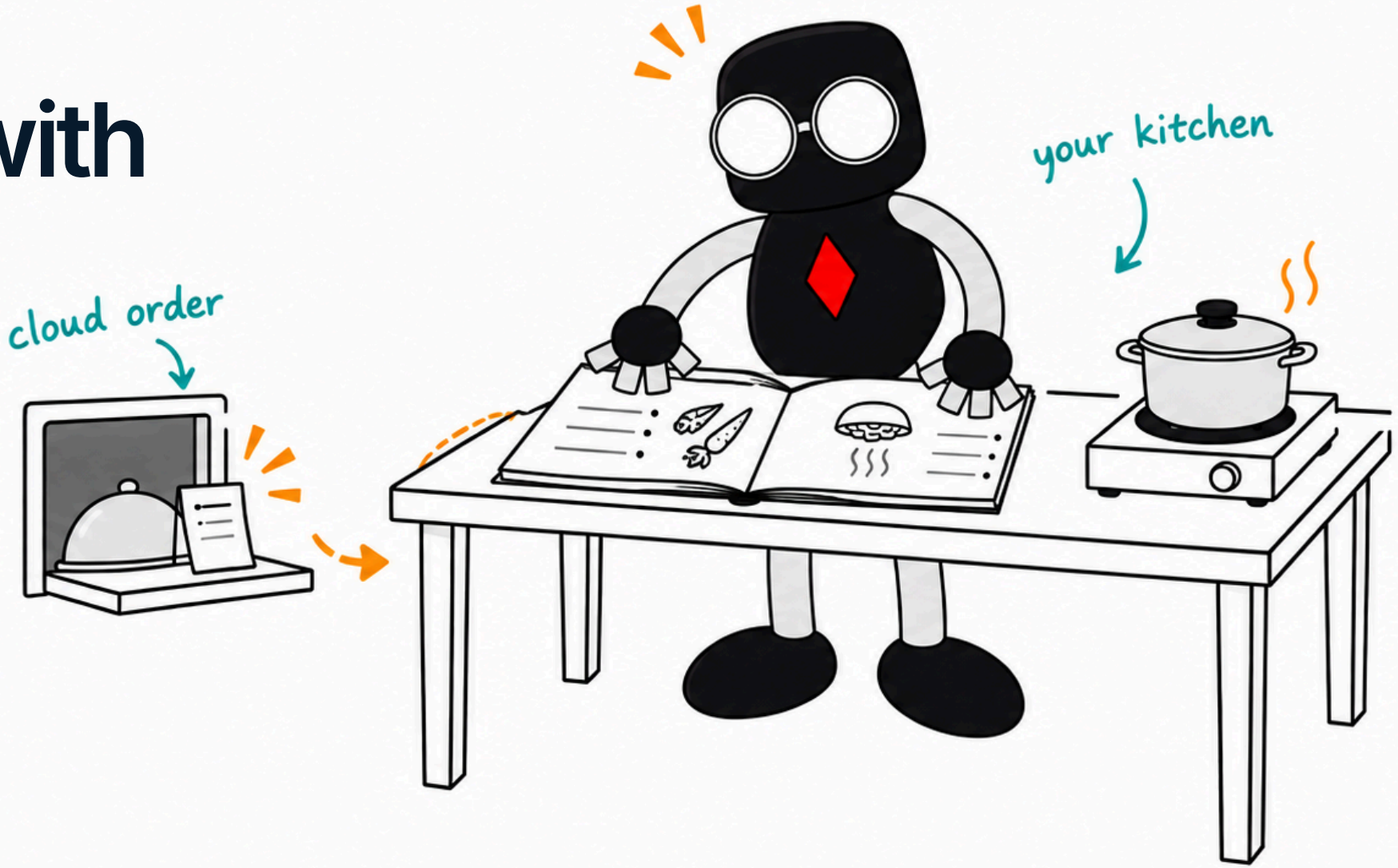
Local AI, explained with food



agentic builders collective

Why cook for yourself?

Local AI runs the model on hardware you control.

No bill for every serving

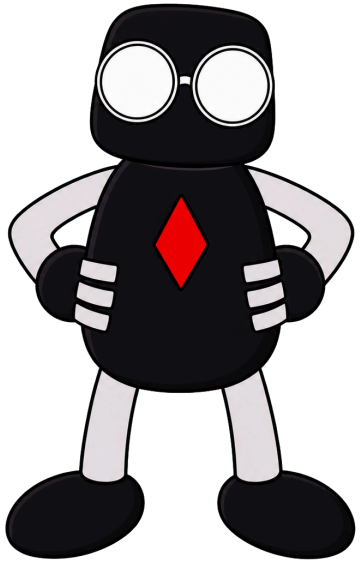
Repeated work without a per-token API bill.

Private ingredients stay in your kitchen

Prompts and files can stay on your device.

Your kitchen stays open

Run offline after setup. Keep the model you choose.



You still pay for hardware, power and upkeep.
Cloud features and connected tools can send data out.

Why local is more practical now

More of the cooking kit is ready to use.

Cookbooks you can download

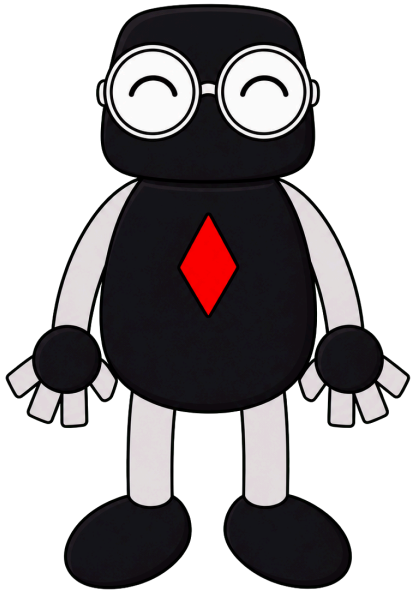
Open-weight models for chat, coding and more.

Less to set up yourself

Apps can download models and serve replies.

More ways to fit the book

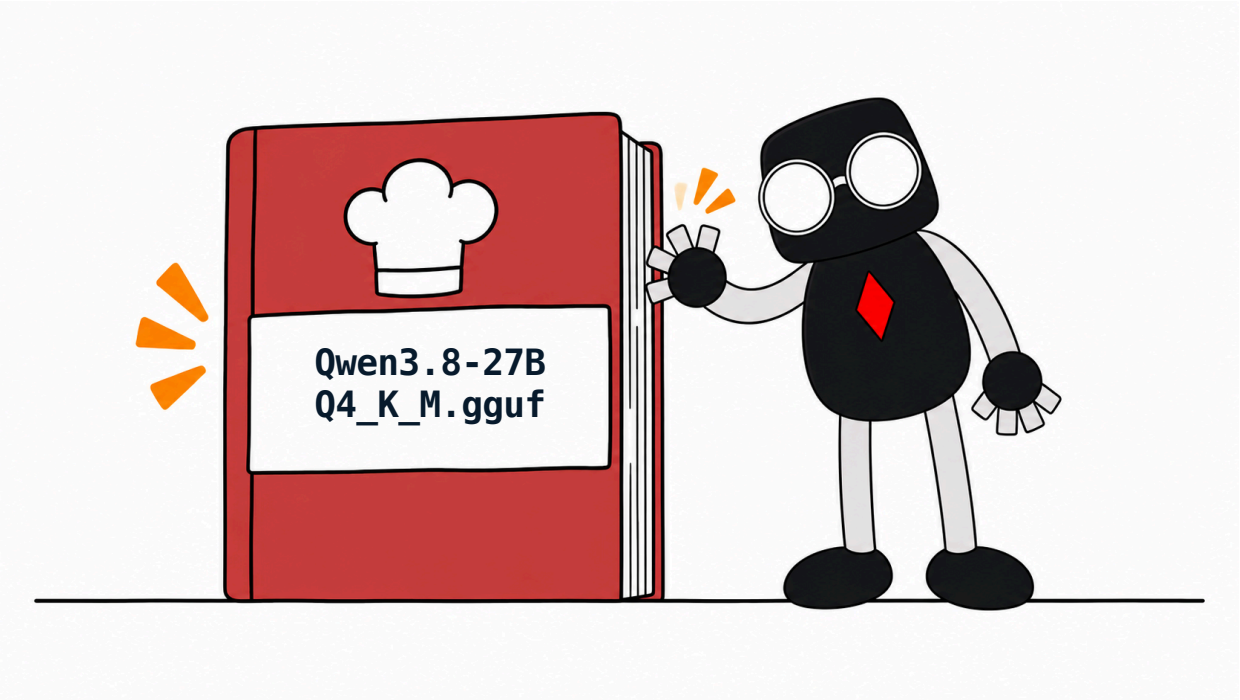
Quantisation shrinks models. Machines with large unified memory hold more.



Quality and speed still depend on the model, hardware and task. Sometimes, ordering from the cloud is worth it.

Read the cookbook label

Qwen3.8-27B-Q4_K_M.gguf



Qwen3.8
Cookbook series and edition

27B
Size of the learned recipe knowledge

Q4_K_M
More agak-agak, less storage

.gguf
Packaging your cooking software can open

Same cookbook. Different setups.

llama.cpp
lean C/C++ engine for GGUF models

Ollama
downloads, manages and runs models

LM Studio
desktop GUI and local server

vLLM
high-throughput GPU serving

SGLang
programmable high-performance serving





Model, runtime and harness

Model: the cookbook

Qwen, Gemma, DeepSeek, gpt-oss

The weights contain what it learned.

Runtime: the cook

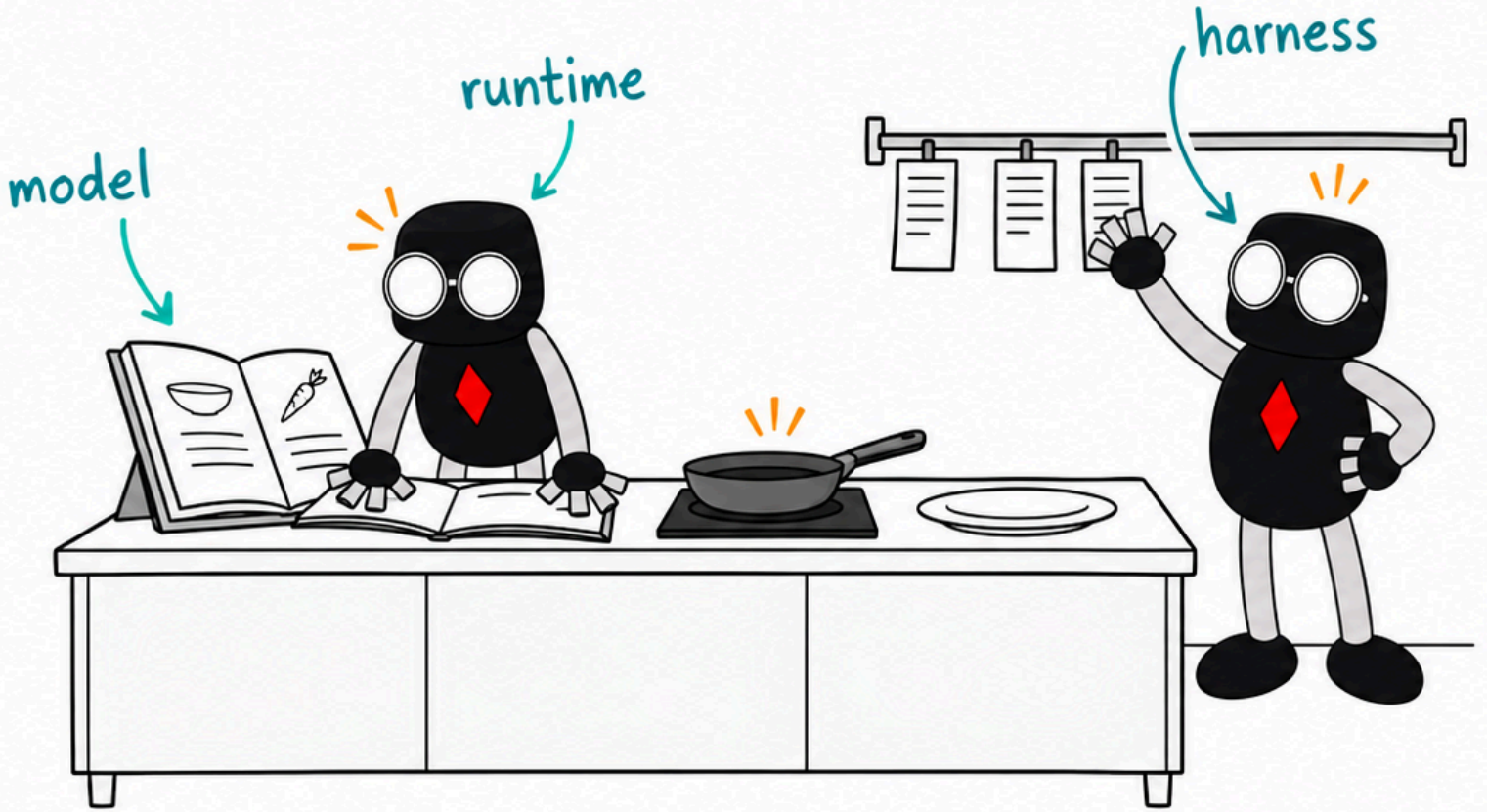
Any setup from the last slide

Runs the model to generate replies.

Harness: the manager

OpenClaw, OpenCode, Pi

Manages tools and the agent’s next steps.



Dense vs MoE

A token is a small piece of text, often part of a word.

DENSE

All weights are used for every token.

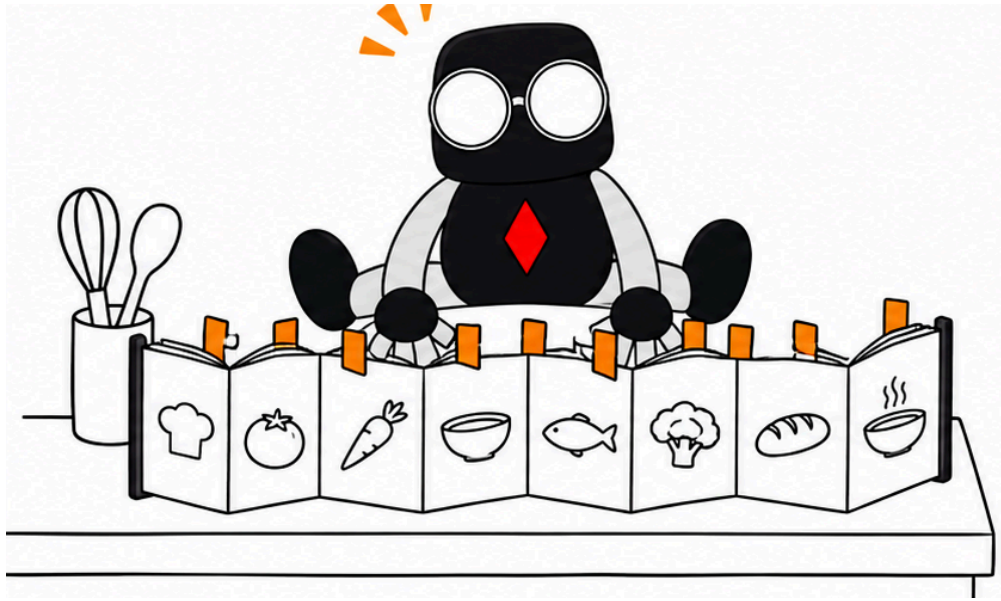
The whole book → each token

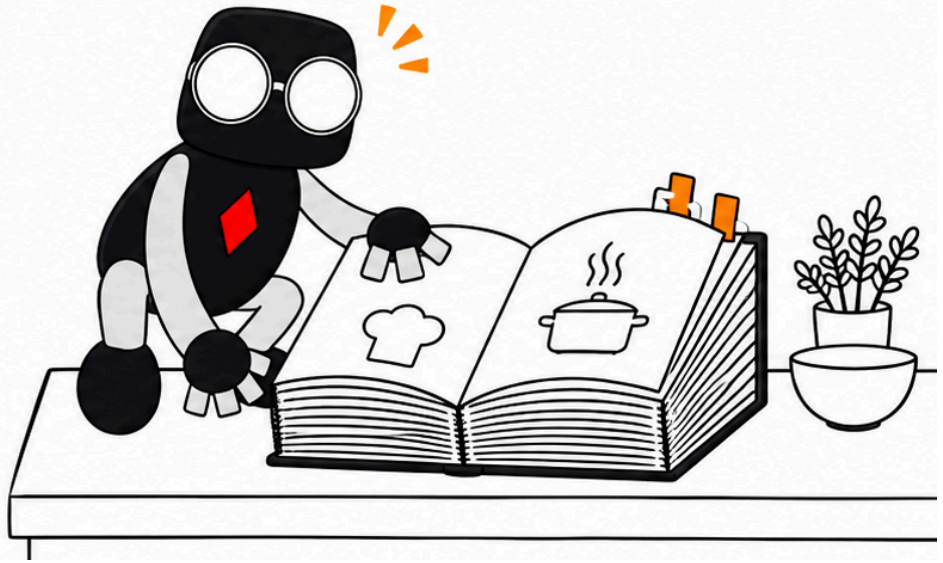
MIXTURE OF EXPERTS

Only selected expert blocks are active for each token.

35B total → 3B active

Qwen3.6-35B-A3B





Total parameters shape what must fit. Active parameters are a rough guide to work per token.

Quantisation

In a recipe, you can measure precisely, or just agak-agak (roughly).

The same learned numbers can be stored using fewer bits.

FORMAT	ROUGH WEIGHT STORAGE	RECIPE VERSION
BF16 / FP16	about 2 bytes per parameter	**fine-grained measuring marks**
INT8	about 1 byte per parameter	**round to the nearest gram**
INT4	about half a byte, plus scales	**choose one of 16 measuring marks**
NVFP4	4-bit values, plus scales	**15 uneven marks, rescaled every 16 values**

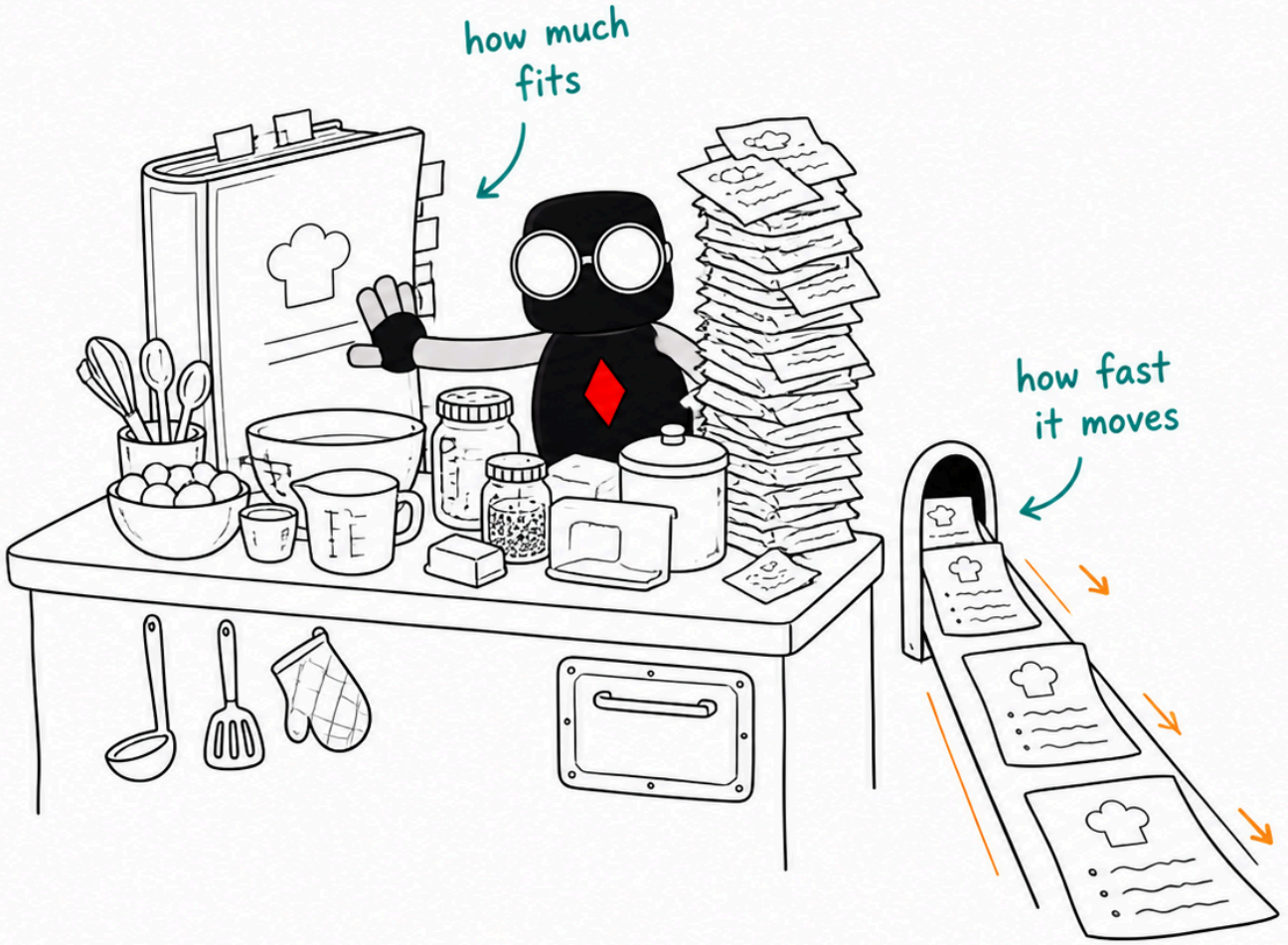
Fit is one thing. Speed is another.

Memory capacity

How much can fit at once

Memory bandwidth

How quickly weights can be read

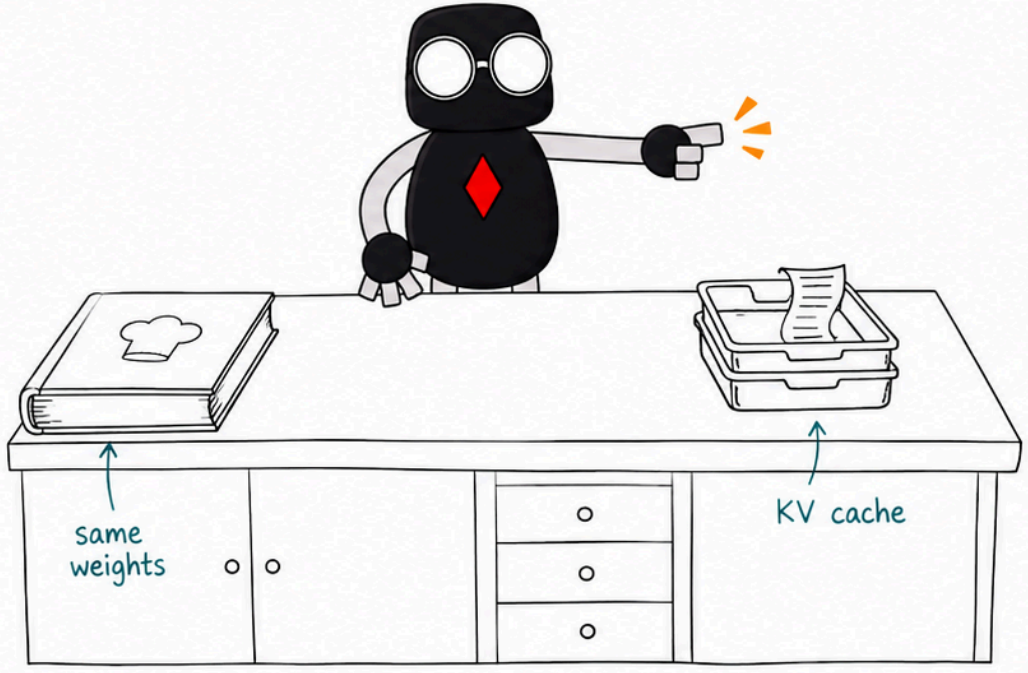


Capacity is VRAM on a GPU, or unified memory on Macs and DGX Spark. Bandwidth is measured in GB/s.

The model fits.
The conversation might not.

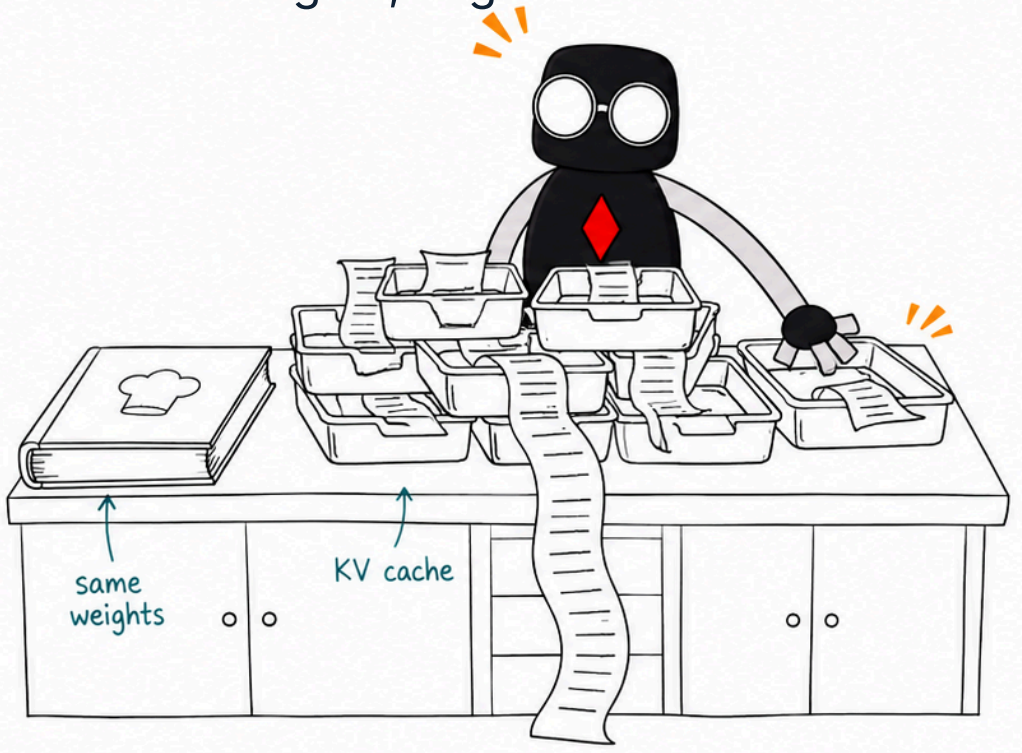
Short chat

Room for context and working memory



Long chat

Same weights, larger KV cache



Context

The text the model can currently attend to

KV cache

Saved keys and values from earlier tokens

What fits in local memory?

These examples may fit in memory, but that does not tell you how fast they run.

Memory target	Model	What the size means
8 GB	**Gemma 4 E4B**	Q4_0 load ≈ 4.5 GB
16 GB	**Gemma 4 12B**	Q4_0 load ≈ 6.7 GB
16 GB	**gpt-oss-20b**	OpenAI: runs in 16 GB with MXFP4
24 GB	**Qwen3.8-27B**	Q4_K_M file 17.8 GB
24 GB	**Gemma 4 31B**	Q4_0 load ≈ 17.5 GB
80 GB	**gpt-oss-120b**	OpenAI: one 80 GB GPU, MXFP4
128 GB*	**GLM-5.3-Flash**	UD-IQ1_S file 93.1 GB; aggressive quant
256 GB*	**GLM-5.3**	UD-IQ2_M file 239 GB; aggressive quant

The model shares memory with its context and software, including the OS on unified-memory systems.

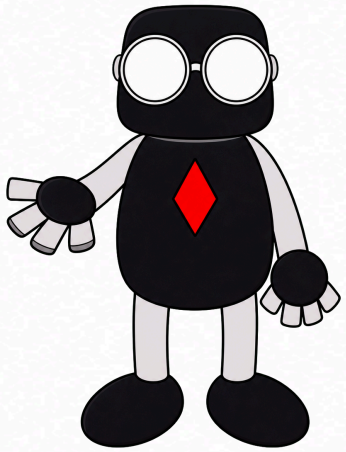
*We have not tested the 128/256 GB setups. Their low-bit quantisation may reduce quality.

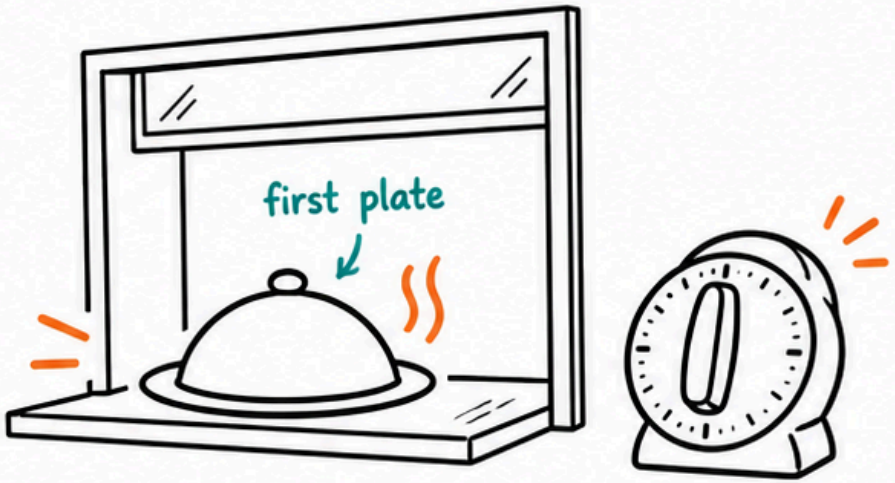
TTFT and TPS

TTFT

Time to first token

Prefill: process the prompt before answering

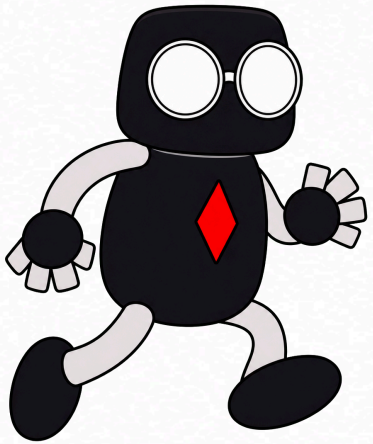


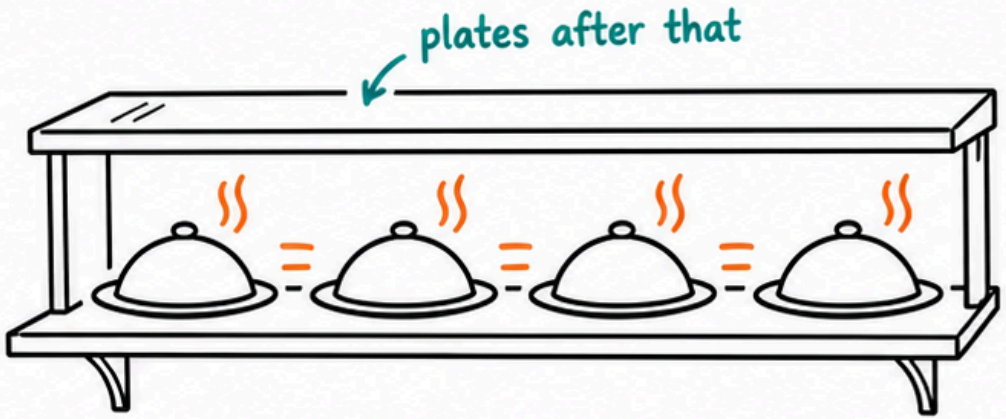


TPS

Tokens per second after that

Decode: generate the reply one token at a time





Speed claims need a recipe

How many orders, what dish, and how much prep?

C1 / C4: concurrency

One / four requests in flight. One agent can launch several.

Cold prompt / prefix cache hit

Prep from scratch, or reuse matching earlier work.

Per-user speed / total throughput

Your plates per minute, or the whole kitchen's output?

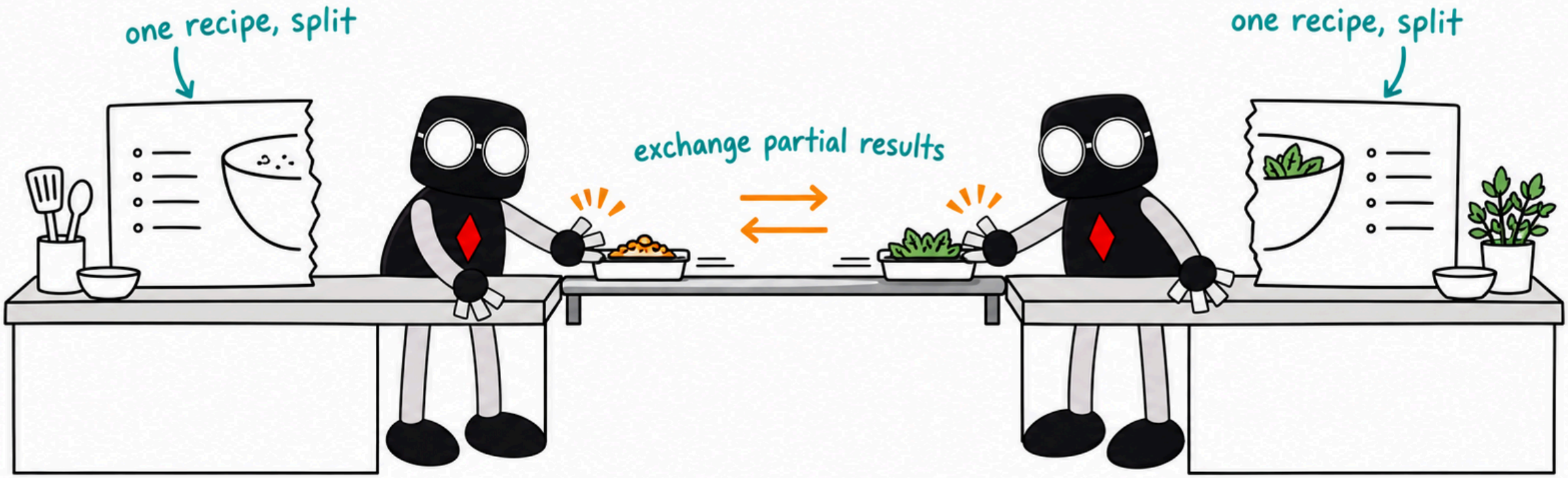
Speculative decoding / acceptance rate

A drafter suggests tokens. The main model checks them.
How many suggestions survive?



A useful benchmark shows how long you wait and how fast the reply arrives, using the same task and settings.

One model across several GPUs

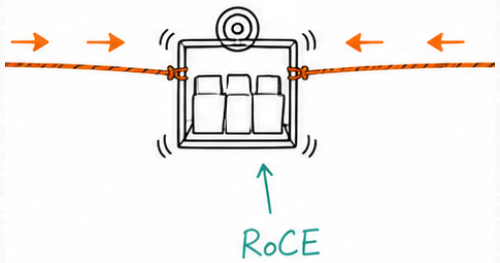


NCCL: software that coordinates GPU communication

TP
Split each layer's maths across GPUs

Within one machine
NVLink or PCIe

Between machines
RoCE or InfiniBand



Fits. Fast. Worth it?

Fits?

Weights, context and working memory

Fast?

Bandwidth, compute, TTFT and TPS

Worth it?

Privacy and control versus effort and trade-offs

